Firmware Block Implementation for the Proto-DUNE II Hit Finding Architecture with the HLS Tool

Project report by J. Horswill

Supervised by K. Manolopoulos

Department of Physics and Astronomy
University of Southampton
United Kingdom

April 30, 2021

Abstract

The Deep Underground Neutrino Experiment (DUNE) is a large-scale international particle physics project based in North America. It will consist of two underground neutrino detectors placed within the path of an intense neutrino beam fired from Fermilab, the furthest of which will be 1300 kilometres from the source, in the Sanford Underground Research Facility (SURF). Objectives of the experiment include the study of certain properties of leptonic CP violation, as well as the analysis of proton decay, and the investigation into black hole formation through supernova neutrino detection[1]. This report focusses on the 'Trigger Primitive Generation' (TPG) cores within the front-end electronics of the detectors. More specifically, the report discusses the hit finder architecture that processes optical data. This data is generated from argon scintillation within the 'Time Projection Chambers' (TPCs), as a result of neutrino-nucleon interactions. The project objective was to replace the existing firmware algorithms written for the detector's 'Field-Programmable Gate Array' (FPGA) boards, with designs synthesised via Xilinx's Vivado 'High Level Synthesis' (HLS) software. C++ code was used to implement the desired behaviour. One of the end goals was to allow the comparison of the new design's performance and resource utilization to those written from scratch in the VHSIC Hardware Development Language (VHDL). HLS has the potential to make firmware design more accessible to a wider range of scientists. This could lead to more people contributing to improving the capability and performance of particle physics detectors and consequently acquiring groundbreaking data at a faster pace.

Contents

1 Introduction

1.1 DUNE Experiment Goals and Context

The DUNE experiment is an amalgamation of many neutrino-physics communities, centred around an opportunity provided by the large investment of the U.S. Department of Energy. This will contribute to a large expansion of the underground experimental architecture of the SURF facility, as well as a megawatt neutrino-beam facility positioned 1300km across North America at Fermilab. This investigation will be groundbreaking for several areas of physics, including neutrino oscillation studies, searches for proton decay, and for neutrino-based astrophysics i.e. black hole and supernova burst phenomena.

1.1.1 The Importance of Neutrinos in the Standard Model

Analysing the properties of neutrinos has led to evidence for physics beyond the Standard Model (SM). Oscillations of neutrino flavour is a well established concept, from which several conclusions can be made. The first is the fact that neutrinos have mass, although unexpectedly small; the sum of the three neutrino mass eigenstates must be less than one million of an electron mass eigenstate [2, 3]. The origin of these small neutrino masses and their oscillations is still not confirmed however, and is up for discussion. Finding the explanation will require access to more precision and detail in the available experimental data. DUNE exists to carry out an investigation of neutrino oscillations to test 'Lepton-sector Charge Parity Violation' (LCPV) as well as the ordering of neutrino masses and the three-neutrino paradigm[1].

CPV is well established in Quantum Chromodynamics (QCD), allowed by the complex-natured Cabibbo-Kobayashi-Mashawa (CKM) matrix[2][5]. One could expect to discover an entirely analogous mechanism to arise in the lepton sector of the standard model (SM), leading to LCPV. One of the collaboration's goals is to determine the rates at which neutrino CP violation occurs within the lepton sector. Another hypothesis proposes that there is a connection between low-energy neutrino physics and leptogenesis [6], which could provide insight into the scenario that led to the baryon asymmetry of the universe. These questions can only be answered by constructing suitable detector equipment; the next section will explore the features of ProtoDUNE's detector's front-end electronics and Data Acquisition (DAQ) system. This will feed into the following section that discusses the HLS firmware blocks that could instruct the front-end FPGA-based data processors.

[1] The quantum mechanical mixing of the three neutrino mass eigenstates producing the three known neutrino flavour states [4]

[2] Used to contain the information on the strength of the flavour-changing weak interaction [5].

1.2 ProtoDUNE DAQ System

DAQ architecture can be split into two parts: the front-end and the back-end. The front-end is comprised of circuitry that acquires the data stream from readout-electronics inside the detector, so that it can process and deliver the stream to the back-end: an offline storing, event-reconstruction computation farm. The focus of this report will be on the front-end, including the detection modules and the interface to the detector electronics.

There are several main factors that influence the design of the DUNE DAQ system. One of which is the hardware and maintenance cost per input channel, which is related to the technology life-cycle of each component. This contributes to the need for long-lasting essential parts such as the Xilinx FPGAs and network components. DUNE is set to run for approximately 20 years; the reliability and maintenance of the detector components is more important when compared to other large-scale projects. The power consumption of the detector modules also requires consideration due to cooling contingencies and the variance in running costs. Modularity of the hardware and firmware blocks is emphasised, so that one change in a sub-module would not affect the rest. This emphasis on customisability carries over to the HLS design process. If the design tools are more abstract and are rooted in commonly used languages such as C++, this will appeal to the vast majority of users (physicists) [7]. For more detail on this matter see section 2. With these objectives in mind, R&D and fast firmware modifications can be a quick and efficient process. Section 1.2.1 will explore several types of detectors used in neutrino beam experiments.

1.2.1 Single and Dual Phase LAr TPCs

The use of TPC modules of Liquid Argon (LAr TPCs) is a matured and popular technique used by several short-baseline[3] neutrino experiments. They are popular for several reasons:

- Argon is a noble element of vanishing electronegativity, which means that electrons produced by ionizing radiation will not be absorbed as they drift towards the detector.
- Liquid Argon is inexpensive yet very dense, increasing the likelihood of a particle interacting by 1000 times when compared to Nygren's TPC design [8]. This is important because the cross section for neutrino-nucleon interactions is very small.
- A neutrino-nucleon interaction causes the Argon to scintillate; a secondary energetic charged particle is generated and travels through the liquid, which causes the release of scintillation photons, the number of which is proportional to the energy deposited by the passing particle [9]. This contributes to the reconstruction of the event.

[3]The baseline refers to the scale of the beam travel distance. DUNE is 'long baseline' as the detector is positioned across the continent from the beam source.

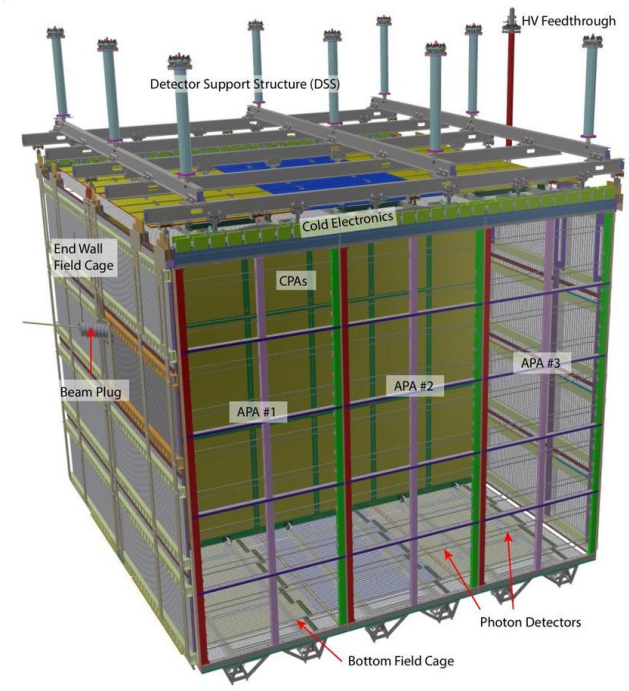

Figure 1: Diagram of the ProtoDUNE single-phase LAr TPC module. This contains two drift volumes, separated by a central cathode plane that is flanked by two anode planes, connected to a field cage that surrounds the entire module. Each anode plane is made of three Anode Plane Assemblies (APAs). These connect and send data to the front-end electronics being discussed in this report [10].

There exists both single and dual-phase versions of these LAr TPC modules. Inside single-phase liquid argon detectors, such as the structure featured in figure 1, the interactions are readout by wires immersed in the liquid and do not require the amplification of the ionized electron charge. The charge is localised using alternative induction and collection wire planes with different orientations. This was tested by ProtonDune-SP at CERN [10].

The dual phase detectors rely on the amplification of ionisation charges in the ultra-pure cold-argon vapour layer above the liquid. In dual-phase LAr TPCs, ionisation charges produced by neutrino interactions drift vertically towards the liquid-vapour boundary where they are extracted into the gas phase, amplified by components named Large Electron Multipliers (LEMs), and collected by segmented anodes. Five photo-multiplier tubes are mounted underneath the TPC drift cage. They detect the Argon (prompt S1) scintillation photons formed from interactions with passing charged particles, as well as the (proportional S2) secondary scintillation light from the electroluminescence of the electrons extracted in the argon vapour. S1 is at least several orders of magnitude higher in detection amplitude than S2 in a tested TPC [11].

The 10 kilo-tonne dual-phase liquid argon TPC is one of the main detector options considered for DUNE. LAr TPCs offer unprecedented 3D imaging capabilities and the functionality of a homogeneous calorimeter. High-resolution imaging is the key to efficient background noise rejection. The next section will describe the interface system which handles the data acquired by the LAr TPCs.

1.3 FELIX

The LAr TPC modules used in the ProtoDUNE II design contain 150 APAs, each containing 2560 wires. There are 10 fibres/data streams in the APA, filled with 256 wires per fibre. The Front End LInk eXchange (FELIX) system is used to receive the data from the APAs, and was first developed within the ATLAS collaboration. Within this system contains the focus of this project: the TPG core, discussed in section 1.4. FELIX is a PC-based networking gateway with the FPGA mounted on PCIe[4] Gen3 cards [12]. In ProtoDUNE II, FELIX interfaces with and manipulates the data from the APA, during which time it is fed through the TPG core within the Hit-Finder (HF) processor.

The readout window per neutrino event is between 3-5.5 ms for a 2.25 ms drift time. The FELIX architecture that handles this influx of events consists of 10 MGT modules. The function of an MGT is to process the APA data stream into computational bits, which are then fed into 10 HF processors, as seen in figure 2. The hits are produced by processing the incoming ADC (Analog-to-Digital Converted) samples in parallel. The MGT output enters the HFs via the data router, which receives multiple-channel, single clock-tick packets. The packets initially exist in a format containing a given sample wave for all channels,

[4]High speed connection ports used in motherboards to connect GPUs, SSD's, wifi chips and more.

and when processed by the data router are converted into a format subsisting of all the samples for a given channel i.e. multiple clock ticks of data. The formatted output is then transferred into 10 sets of 4 TPG block processors; one set for each MGT output. The processor outputs are multiplexed[5] by the arbitrator into the Central Router (CR) interface [7, 13] as seen in figure 3. Section 1.4 will explain the TPG unit in further detail.

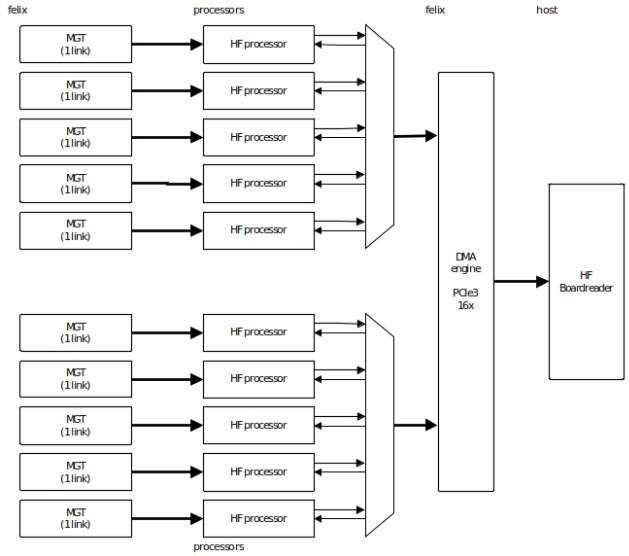

Figure 2: Overview of the Felix architecture in ProtoDUNE [14]

1.4 The TPG Core

Each TPG unit processes the data from 64 APA wires, and so there are 4 units for each core[6]. The TPG data consists of a header and a payload. The header determines the source wire, the time stamp, and the origin of the data. The payload is 64 bits of ADC sample data for hit finding. There is an AXI4-stream protocol for the unpacking and reordering of data in the TPG block, featuring 6 general signals for various instructions in each component:

1. `tvalid` denotes if the bus data is valid for processing.
2. `tkeep` goes high when the incoming data should be processed.
3. `tuser` is a flag that denotes the end of a packet frame.
4. `tlast` indicates the end of a packet.

[5] Combined into a singular signal output.

[6] $4 \times 64 = 256$, the number of wires in a fiber; ten fibers summed over ten MGTs accounts for all wires within the APA (2560).

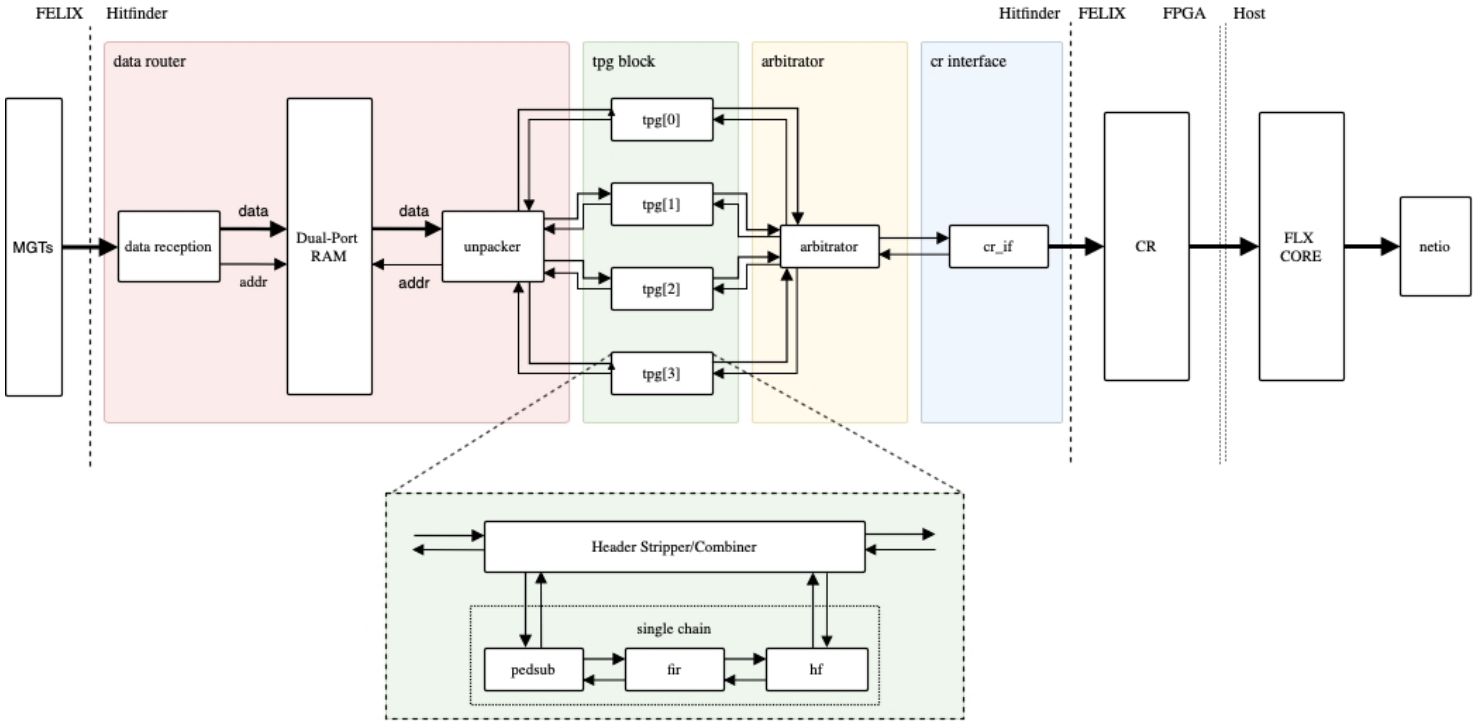

Figure 3: Hit Finder Firmware Architecture [14].

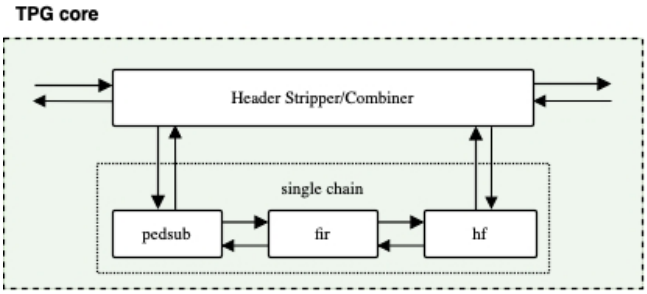

Figure 4: Focus on the HSC and the sub-block chain [14].

5. `tready` is used to apply back-pressure if the processing units ahead cannot process the data fast enough (all throughput is frozen).

6. `treset` goes high when the entire process must be reset and restarted, sometimes used when an error in the FELIX interface occurs.

Within the TPG core is a mechanism that strips the packet header: the Header Stripper/Combiner (HSC), featured in figure 4. The header is sent to the output of the block. Meanwhile, the remaining 64 ADC samples and their associated AXI4-S signals are processed by the PedSub (pedestal subtraction), FIR (finite impulse response), and HF sub-blocks in the chain. The data router is then signalled to send the next packet forward for input.

The objective of this project is to redesign the firmware algorithms for the TPG sub-blocks in Xilinx's Vivado HLS software suite. This is to acquire a comparison of the performance and resource usage of the HLS design with the existing firmware written by VHDL engineers. Section 2 goes into more detail on this matter.

2 HLS

2.1 Background

High-level synthesis (HLS) is an automated design process that interprets an algorithmic description of a desired behavior and creates digital hardware that implements that behavior. It was initially designed to increase the levels of abstraction for design methodologies, forced by the growing capabilities of silicon technology and the increasing complexity of their applications. Beginning in the 1950s, assembly languages were introduced and gradually evolved to high level languages with improved compilation techniques, obeying the intuitive rules of grammar, syntax and semantics. Gate-level simulations appeared in the 1970s, and in the 1980s progress led to Hardware Description Languages (HDLs). A HDL is a programming language that describes the structure and behaviour of electronic circuits, and more commonly digital logic circuits [15]. HDLs look like

programming languages such as C, but the main difference is that they include the notion of clock timing. The most commonly-used languages are Verilog and VHDL, the latter of which is used in the DUNE firmware framework. HLS allows for a connection between software languages and hardware implementation. In the case of section 3.1 this is achieved via VHDL firmware.

Synthesis begins with a high-level specification of the problem, where behavior is generally decoupled from low-level circuit mechanics such as clock-level timing. Early HLS explored a variety of input specification languages, although recent research and commercial applications generally accept synthesizable subsets of ANSI C, C++, SystemC and MATLAB. C++ was used to design the algorithms featured in section 3.1. This input source code is analysed, architecturally constrained, and scheduled to transcompile into a register-transfer level (RTL) design in a chosen HDL, which is in turn commonly synthesized to the gate level by the use of a logic synthesis tool [16].

RTL is a design abstraction which models a synchronous digital circuit in terms of the signal flow between registers and the logic operations being performed on said signals [17]. Hardware registers are circuits composed of flip flops: a sub-circuit that has two stable states used to store state information. They are a basic storage element in sequential logic, possessing the ability to read or write multiple bits at a time and use an address to select a particular register, used as an interface between the software and the computer peripherals [18]. RTL allows for circuitry simulation and digital analysis, and is used in HDLs to create high-level representations of a circuit, from which lower-level representations and ultimately actual wiring can be derived. RTL clock cycle data is given in Vivado HLS performance estimate data, as it can read how long a certain block, loop or function in the code took to complete; this is called the latency. Latency is a crucially important concept in RTL design, as it can make the difference between synthesizing a malfunctioning system or obtaining a synchronised circuit. Before the firmware block in question can be discussed, the design flow within Xilinx's Vivado HLS development software will be explained to provide context for the generation of the algorithm.

2.2 Vivado HLS Design Flow

The design flow when approaching the synthesis of a C algorithm is as follows:

1. Compile, simulate, and debug the C algorithm.

   This involves validating that the C program correctly implements the required processes. In order for this to occur, there must be a test bench written in C that includes the top-level function called within a simulation script, tasked with recreating the conditions of the implemented firmware, hosted within `main()`. This typically takes more time to design than the function to be synthesized.

2. Synthesize the C algorithm into an RTL implementation, optionally employing user optimization directives.

This allocates hardware resources such as Block Random Access Memory (BRAMs), data buses, look up tables (LUTs) [7], digital signal processors (DSPs)[8] and registers. Each operation is scheduled to a certain clock cycle and bound to a functional unit within the board. Variables are bound to storage elements, and the RTL architecture is eventually generated[17]. See section 2.2.1 for more information on directives.

3. Generate comprehensive reports and analyse the design.

   After C synthesis, output files are stored in a directory which contains folders for each RTL output format, and a report folder. The report folder contains files reporting on the performance and design corresponding to the top-level function and each individual subfunction. The analysis tab allows for cycle-by-cycle insight into the timing of logic operations and the FPGA resource consumption that they require. This is the best opportunity to analyse the effect of applied function directives and `pragma`[9] commands.

4. Verify the RTL implementation using a pushbutton flow.

   This is a post-synthesis validation process that compares the synthesised RTL and C programs to see if they generate the same output; this is called C/RTL co-simulation.

5. Package the RTL implementation into a selection of 'Intellectual Property' (IP) formats.

   This generates the verified VHDL script that can be synthesized in the Vivado project manager into an implementable design. Before this can occur, a VHDL wrapper is written to connect the IP block's I/O ports to the previous and following blocks, ensuring signals are being properly read and written. This is especially important for the AXI4 stream protocol discussed in section 1.4, as sufficient handshake and boolean signals must enter and leave the block for the AXI4 interface to be able to validate and manage the incoming and outgoing data samples. This can be tested in Questasim[10] to provide a waveform analysis of every port and internal register involved. This is an important verification step in the design process, as the performance of an algorithm within the HLS environment can be vastly different to a VHDL simulation [16].

[7] LUTs are arrays that replace runtime computation with a simpler array indexing operation, saving processing time [19]. FPGAs employ reconfigurable, hardware-implemented, lookup tables to provide programmable hardware functionality.

[8] DSPs are specialised microprocessing chips which are usually employed to measure or manipulate analogue signals. In the case of this project, they are used to perform rapid addition and multiplication calculations [20].

[9] Pragmas are directives that are stored within the source code, rather than in an external `.tcl` file

[10] Software dedicated to simulating HDLs. This is typically used in collaboration with the Vivado project manager environment, where full VHDL block chains can be synthesized and implemented within an FPGA.

When writing HLS source code, there is punishment for using modern modules and storage entities; a simplified, classical style is most easily synthesized. An example of this is that storage arrays must have pre-allocated memory, with a determined number of elements at compile time (ruling out C++ vectors). Additionally, array manipulation modules such as `std::copy` and `memcpy` have very limited functionality or none at all, depending on the circumstance. These are some of many restrictions one faces when writing the desired behaviour in C++. Ultimately the code must be portable.

The performance of the logic operations come into question when constrained by a certain latency in order for the block to operate as required. The clock period must simultaneously stay below $1/f_{target}$, which in the case of the TPG core, the target frequency $f_{target} = 250$ MHz, meaning the clock duration must stay below 4ns. However, it must tick for long enough so that the block operations can occur within the desired latency. Performance comes at a trade to resource cost, as pipelining (parallelizing and rearranging operation initialisation to decrease the latency and input interval) employs more FPGA elements, which are a finite resource. One way to reduce this cost is to assign accurate bitwidths to variables. Using standard C data types for hardware design results in unnecessary resource usage. Operations can use more LUTs and registers than needed for the required accuracy, causing delays that may exceed a given clock cycle, increasing the overall latency. Smaller bit widths (e.g using 17 bit data types for 17 bit multiplication rather than the standard C 32 bit data type) result in hardware operators which are smaller and faster. More logic can then fit in the FPGA which can operate at a higher clock frequency. The next section will discuss the management of performance and area when choosing HLS design guidances.

2.2.1 Optimisations and Analysis

Directives act as a low-level guidance for the HLS tool, influencing the types of hardware used and the methods of implementing the source code operations [21]. A very useful example of this is the division of arrray elements into registers using the `array_partition` directive, allowing for faster data access at the cost of increased resources.

Different optimization directives can be applied before C synthesis; one of the most crucial is the `pipeline` directive. This instruction forces code tasks to execute non-sequentially, allowing the next execution of a task to begin before the current one finishes, visualised in figure 5. One can infer from the figure that dependencies between two different operations can affect software's capability to pipeline the entire function, meaning the II cannot equal one until code changes occur and dependency directives are implemented. It can be deduced that if back pressure went high during the operation of the pipelined function `func(m,n,o)`, 3 cycles (the number of active function 'layers' at any one time) would pass until the block is fully paused. This would not provide instantaneous behaviour required, hence the requirement for II = latency, which if we require a fully pipelined design, latency must equal one.

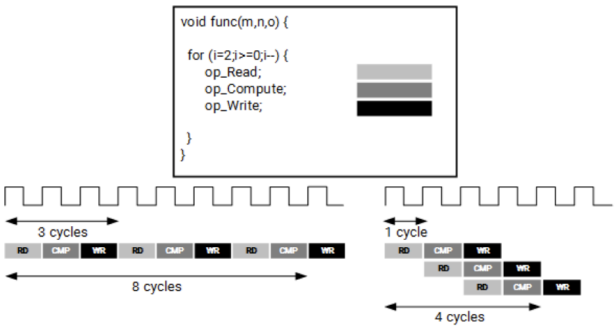

Figure 5: This figure contains an example showing how read, compute and write operations can happen in parallel to achieve higher throughput overall, using the `pipeline` directive.

In order to develop a fully pipelined block, code changes must be made to remove internal dependencies such as multiple store and load operations on registers and BRAMs (or any storage resource) at very different times during the operation of the block. This can be remedied by reducing the number of times to which a register is written via logic branching. This can then be further improved upon by applying `dependence` directives that identify false dependencies between arrays or variables, and allow the function operation to be even more streamlined. The user must take caution with instructing HLS to recognise a false dependency for a variable where there could in fact exist a true dependency. This can lead to C/RTL co-simulation failure and block malfunction.

Functions called within the top function can employ the `inline` directive so as to reduce overhead and minimise pipelining issues when calling sub functions. Not only can performance be adjusted by directives, but the HLS tool can also specify a limit to the resources used by the block, allowing for a balance between speed and area usage. Additionally, more customisation opportunities arise from the ability to select a block's I/O protocol to make sure the final design can be connected to other hardware blocks with the same framework.

There are several viable ways to achieve an optimal directive configuration for user's purposes, but the best way to gauge the effect of adding directives in the Vivado HLS tool is by analysing the cycle-by-cycle display of operations within the synthesis reports. One can analyse a number of aspects surrounding the performance estimates of the block, perhaps the most important of which is clock timing. This is the fastest achievable clock frequency before the block ceases to function. This must meet the target to synchronise with the surrounding firmware blocks.

Secondly, the block latency and initiation interval[11] (II) must be reviewed, since the internal operations of the block must have enough time to complete

[11] The number of clock cycles before new inputs are processed.

before the next input is processed. Without any pipeline directives, the latency defaults to one clock cycle less than the II so that this can be achieved. In many cases the top function logic and even the input data types must be rewritten several times before the design reaches its performance objectives. See 3.1.3 for an example of this.

Finally, the utilization estimates are key to making the trade-off between latency and FPGA areas. Limited budget and gate area means that unlimited resource usage is not an option, therefore this must be tracked.

Bear in mind that some of the observations made in this section are specific to the design of detector signal filtration algorithms, and is not necessarily optimal for all HLS design projects. There are many more directives and design techniques that can be found in the Xilinx documentation: UG902. Now that a sufficient introduction into the design process has been given, the next section will explore the key work outlined in this report: the firmware algorithms employed by HSC block chain first mentioned in section 1.4.

3 Development and Findings

3.1 Pedestal Subtraction Algorithm

The pedestal subtraction algorithm was the first mechanism in the project to be redesigned in HLS. It removes each channel's base background noise from the incoming event data, so that the following FIR and HF blocks are dealing with the isolated optical signals from the neutrino events occurring within the detector. This should not be confused with the reduction of random noise of the isolated signal, which is taken care of later in the block chain. It is implemented within first sub-block in the TPG core, `pedsub`, which is fed by 64 sample words[12] from the Header-Stripper/Combiner shown in figure 4. Before the performance and resource utilization can be compared to the original design, it is necessary to demonstrate the algorithm's mechanisms.

3.1.1 Algorithm Operations

The input packet initially contains 70 sample words. However, the first 6 16b words are stripped from the incoming samples by the HSC and the remaining sample are sent to the `pedsub` block. There are three important variables used to achieve the reduction of background noise: the pedestal (or median), the accumulator, and the incoming ADC sample. The pedestal acts as the average background signal for a given channel, used to subtract from the data samples to produce the 'true' isolated signals. The pedestal and accumulator are adjustable values that must be set to an estimate before any samples have entered the block. When the pedestal is greater than the incoming ADC value, the accumulator is decreased by one, and if the opposite is true, the accumulator is increased by

[12]Known as a data packet, each sample contains an input ADC value and the associated AXI4 signals.

one. When the accumulator reaches a limit of ±10, the pedestal is increased by one or decreased by one respectively, and the accumulator is reset. See figure 6 for an illustration of how the algorithm manages signal noise.

Connected to this block, or implemented within it, is a State Save/Restore (SSR) mechanism, which is used to save these adjustable variables at the end of a packet. For the HLS design that relies on an external SSR mechanism, this is implemented via a VHDL source file connected to the `pedsub` wrapper, that receives and feeds values to the pedestal subtraction logic. This can also be managed internally, the method for which is discussed in section 3.1.3.

The SSR functionality enables a variable restoration every time a packet enters a previously-processed channel, allowing for two constantly fine-tuned arrays of median and accumulator values to be used for the remainder of the detector run time. Therefore once `tlast` goes high, the final pedestal and accumulator values for the current channel are saved and those stored for the next channel are retrieved from the SSR. Every 64 packets, a saved value is again restored for the next packet entering the same channel. This is because packets are processed for each channel sequentially, beginning from 0 and reaching channel 63. This means that once the `pedsub` block has run for every channel, there will be a better background noise estimate and therefore more accurate ADC output data from the pedestal subtraction. Eventually, with a large enough flux of packets, one would obtain an optimal pedestal value appropriate for the background noise of each input channel.

3.1.2 Redesign of Pedestal Subtraction with External SSR

The development and debugging of the `pedsub_HLS` block led to a number of insights for future use of high-level synthesis when designing detector firmware. The first thing to note is that when developing an algorithm, it is advisable to start with a simplified version of the target function. This is because complex functions are especially prone to invalid signals and incomplete output, which can be difficult to debug when trying to trace back the unfamiliar proxy signals in machine-written VHDL code.

One could argue that it is better to fix output issues within the HLS environment rather than try to treat 'symptoms' of the problem by editing the IP block file. There are limited Questasim warnings when building a simulation project to test the generated HLS IP, and observations can be made from the output signal waveform, but regular trial and error must be employed in order to target the source of an issue. This is a drawback of HLS IP, since the post-synthesis design lacks transparency when compared to writing HDL projects from scratch. One of the best approaches to deal with this is to slowly add layers of complexity, and each time an addition is made, one can test the functionality in Questasim and observe successful synthesis in the Vivado project manager before adding more features.

Contingencies must be included within the wrapper to account for a disparity between the IP block and wrapper port's bits widths. Logic may also be required to compensate for the timing of handshake signals outputs, such as with the

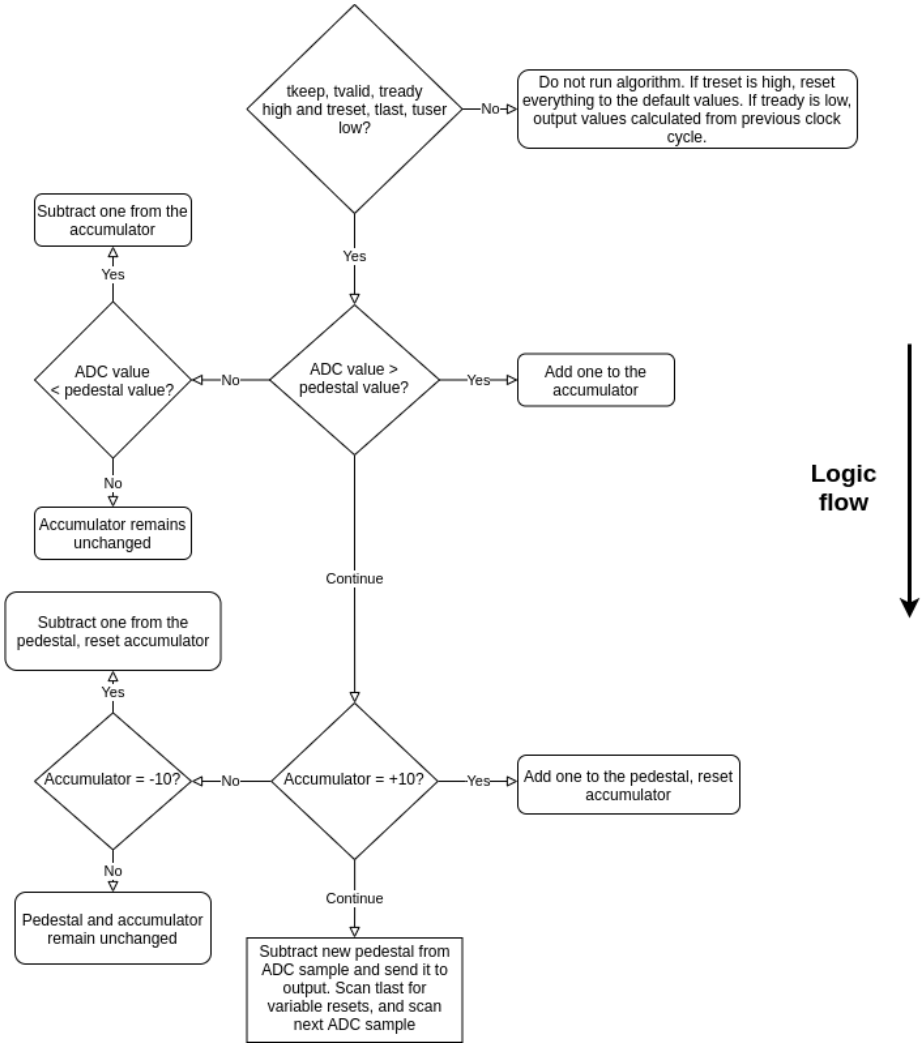

Figure 6: Flow chart detailing the logic flow of the pedestal subtraction algorithm, demonstrating the adjustment of the pedestal and accumulator values, as well as the occurrence of the subtraction from the ADC value.

delayed internal variable restoration when `tlast` goes high, discussed below. This links into the inclusion of static variables that save internal values between function calls.

The Vivado HLS synthesis tool recognises a static variable in the same way a C++ compiler would, generating a static storage area for that value. An example of their use in the pedestal subtraction algorithm is the storage of a clock cycle counter that starts when the `tlast` signal output goes high. This static variable is saved between multiple function calls until it reaches three cycles. When this happens, the statically-stored pedestal and accumulator registers are restored to their SSR values for the next channel. Static variables are a crucial tool when a self-sufficient block is required and externalised I/O management blocks are inconvenient.

However, one main issue faced earlier in the project was the long-term storage of the pedestal and accumulator: static variables were wiped between incoming packets. It can be seen from the `pedsub_HLS` Questasim waveform that this variable wipe occurs five clock cycles after the following packet's `tvalid` signal goes high. A separate SSR storage array mechanism had to be developed for restoring values to a given channel in the long term[13]. In the preliminary report, HLS was deemed inconvenient when developing a long-term SSR mechanism; the block being called every clock cycle means that new declarations are reset very frequently, unless they are static. Further investigation needed to be made into the optimal approach, and this was found and utilised in the designs featured in sections 3.1.3 and 3.2.3.

Returning to the `tlast` mechanism, it can be discerned that the automatic static variable wipe happens too late; the restoration of values 5 clock cycles after `tvalid` goes high means that the first 5 subtractions occur using the previous packet's pedestal, which leads to 5 invalid results for the next channel. A manual restoration of the internal values must occur before `tvalid` goes high for the following packet. However, this restoration must be after the previous packet's `tlast` output goes high; the `tlast` input is not used as this would still truncate the last value. This restoration window is prioritised in order to avoid invalidating the previous packet's ADC output values. It was found that restoring 3 clock cycles after the tlast output goes high allows for the restoration of the pedestal and accumulator to occur from the existing SSR mechanism. This is an example of how chronology can affect the output of a block in unpredictable ways.

Another clock timing concern is the required instant response to back pressure signal `tready`. Traditionally, the block would react too late as a result of the latency delay from input to output. This issue can be circumvented in the case that the block takes a maximum of 1 clock to complete, hence all functional designs mentioned in this report have a full operational time of 1 clock.

On input, the first logic branch depends on the `tready` input being high or low. If `tready` is low i.e. back pressure is active and the block operation

[13] i.e saving and then restoring packets 64 packets later, or a minimum of 4480 clocks after the initial packet enters a channel

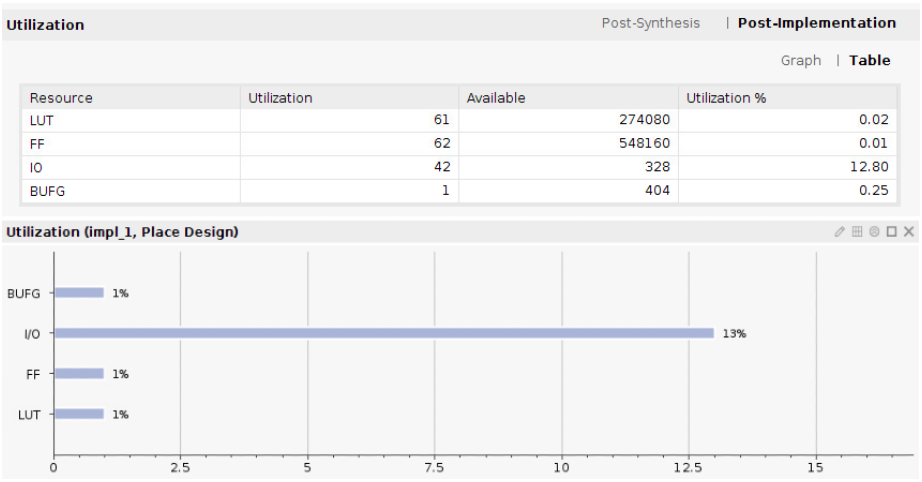

Figure 7: The resource utilisation for the original pedestal subtraction block including the exact figures above and percentage utilization below. All resource statistics (figures 7-15) were taken from the Vivado Project Manager post-implementation report summary for each block. Please note that for all resource bar charts, utilization percentages above 1% have been rounded to the nearest percent. Utilizations below 1% are rounded up to the nearest percent.

requires pausing, the mechanism will continuously force an output of the same values that were sent through 1 clock cycle before the back pressure went active. This will cease only when tready goes high again. The desired behaviour can be achieved by constantly saving the previous clock's values and then restoring them to the output once the `tready` input goes low. However, if the block latency was greater than 1 clock cycle and the II = 1, back pressure would occur halfway through the operation of one function call, meaning the block would not react in time and erroneous outputs would be sent through to the FIR.

The final timing concern is ensuring that the outputs are transmitted at the same pace that the inputs are received, which can be achieved with the `pipeline` directive discussed in section 2.2.1 as well as careful optimisation of the logic. The next section will discuss the disparity in resource cost between the original design and the new HLS design.

Performance and Resource Utilization Comparison

The HLS block was forced to perform at a max latency of one clock cycle with an II equal to the latency, meaning the outputs arrive one cycle after the inputs and the inputs are accepted every clock cycle. This can be compared to the performance of the existing algorithm which has a latency of two and an II of one. The synthesis report shows the block has a clock cycle of 3.6±0.5

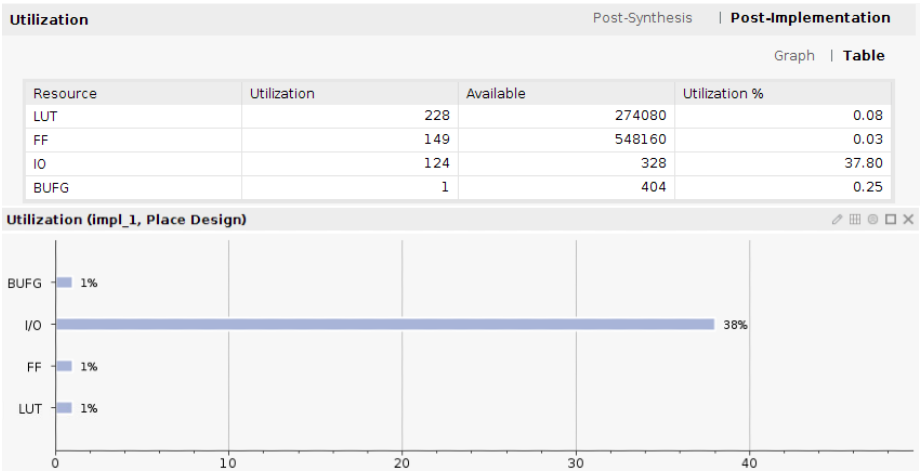

Figure 8: The resource utilization of the new HLS block, including exact figures in the statistics table above the graphical representation of the percentage utilizations.

nanoseconds, and an RTL co-simulation confirmed the block satisfies the target frequency of 250 MHz and a confirmed latency of one clock cycle. The HLS block outputs the same values as the previous design, with a 1 clock cycle delay due to the difference in latency. This was verified by a ModelSim simulation comparison between the blocks, and several model software output files.

When comparing resource utilisation between the `pedsub` blocks, detailed in figures 7 and 8, the most important components to compare are LUTs, FFs, and the input and output (IO)[14] resources for the FPGA target[15]..

The utilization tables show that the new design requires 0.06% more LUT usage when compared to the total LUTs available, which although comes to a 274% increase in total LUT usage for the block, is negligible when compared to the total available. Similarly, the FF usage for the HLS design shows an 140% increase from the original, but only a 0.02% difference in total usage of all available FFs. The disparity in LUT and FF usage can be explained by the additional internal logic and registers required to implement the delayed tlast restore mechanism and the instantaneous response to tready. The disparity is also derived from the halved latency of the HLS block when compared to the original block. This disparity could be dealt with by further investigation into the AXI4 Stream Protocol directives described in section 3.2.1.

Regardless, the difference in IO resource usage is not negligible, due to the large number of scalar and pointer type ports in the `pedsub_HLS` design compared to the original. There is a 25% increase in the FPGA target IO usage of

[14]These are physical structures that allow the connection of an FPGA target to other devices within the system.

[15]A programmable area or chip on the FPGA used to implement a digital circuit, in our case designed via HLS or directly through VHDL.

the new design when compared to the total available, and a 195% total increase in the number of IO resources used. This can be remedied by removing unnecessary ports and incorporating multiple ports as members of a C++ `struct` type input, which can be manipulated further to reduce IO usage. This method is used in the designs featured in section 3.1.3, 3.2.2 and 3.2.3. The following segment will include discussion on the pedestal subtraction HLS block with built-in state save/restore behaviour.

3.1.3 Pedestal Subtraction with Implemented SSR

The pedestal subtraction algorithm `pedsub_HLS_SSR` is a HLS synthesized block including a memory-based SSR mechanism. The basic processing of the accumulator, pedestal and ADC values occur in exactly the same way as the design in section 3.1.2. However, in order to accommodate the performance deficit that came with storing and loading from a memory, the logic branching had to be adjusted. Additional temporary registers had to be utilised to avoid pipeline dependency issues. Arbitrary bit-width signed data types were used so as to reduce resource usage, and remove the need to truncate or zero pad input or output signals in the wrapper. Sub functions called within the top function were inlined and pipelined to reduce overhead. C++ `struct` data types were used as the function arguments (i.e. the block ports) with the original AXI4 stream signals as the struct members. For each new design in this project, a new testbench was written to validate the behaviour of the top function in the HLS environment. As previously mentioned, testbench scripts often took more time to write than the top functions themselves, but ultimately provided a priceless quality control for implementing new features.

Memory Utilisation for the SSR Mechanism

The biggest change between `pedsub_HLS` and `pedsub_HLS_SSR` is the use of two global C++ arrays, defined outside the bounds of the top function, to save and restore the pedestal and accumulator values. These arrays can be accessed by the top function and act as permanent storage entities during runtime, solving the SSR problem faced in the preliminary report. HLS automatically implements these entities as two BRAM components. BRAMs support a very limited data access rate. This can cause problems during synthesis of the RTL files and needs to be worked around in some cases. The maximum access rate that a BRAM can support in this case is twice per clock cycle: one read operation and one write operation.

Fortunately, the only time `pedsub_HLS_SSR` needs to access each memory is to read and write once at the end of the `tlast` delay period. Therefore no pipelining or memory access issues were met at synthesis; the saving and restoring to the memory occurred seamlessly. However, in order to achieve latency = 1, these arrays had to be fully partitioned into registers to increase the speed that each memory array can be accessed. These registers were generated as 'power-on initialisations' similarly to static variables, except the partitioned global arrays

are not wiped between packets. In examples such as the `fir_HLS_SSR` block discussed section 3.2.3, the memory is too large to be partitioned, and so BRAMs were necessary to store the SSR values whilst avoiding massive LUT and FF usage from full array partition. Different conditions call for varying memory implementations.

The `pedsub_HLS_SSR` function had to be streamlined extensively in order to fit all operations into one clock cycle, which as mentioned before is a necessary condition for successful back pressure behaviour generated internally, rather than via directives. In order to correctly save and restore the pedestal and accumulator, a static channel counter was used as a memory index between the values of 0 and 63. During the `tlast` delay period at the end of each packet, the current channel's pedestal and accumulator values were saved to the SSR memory 2 clock cycles after `tlast` went high, and the next channel's values were restored on the following clock cycle. This avoided pipelining dependency issues involving the partitioned arrays and the static accumulator and pedestal variables. The channel counter would also increase after the end of each packet.

The `treset` functionality was removed in order to provide more space for the SSR. This reset mechanism was proven obsolete since all storage entities were automatically wiped by the external FELIX system at the beginning of a data run. The only issue encountered in this case was making sure non zero starting values and booleans were set correctly at the start of a simulation, since these are prepared erroneously by the automatic restart. This can be achieved by declaring and defining global entities as their starting value on the same script line, which are then changed after the first clock cycle.

Performance and Resource Utilization Comparison

The main benefits of the redesigned pedestal subtraction block with an in-built SSR are increased performance (faster throughput) and the convenience of a self-sufficient block, with no external systems responsible for the memory access behaviour. The clock period minimum of this design was shorter than the previous algorithm, reported at 2.966 ± 0.5ns. The simulation output for multiple packet waves with a randomly oscillating back pressure signal matches the model output files used to test the original pedestal subtraction SSR block.

In contrast to the design in section 3.1.2, the IO resource utilization of the HLS block was more efficient than the from-scratch design, with a 16% decrease from the original design's IO usage and a significant 3.36% decrease when compared to the total IO resources available. These statistics can be found in figures 9 and 10. `Struct` type function arguments and more efficient port generation is responsible for this. However, the resource benefits end here.

The LUT usage of the HLS design is significantly higher at 481% of the original, or a non-negligible 0.26% usage increase of total LUTs available, which can be attributed to the LUTs required to process the operations on 128 registers for the pedestal and accumulator SSR arrays. FF usage increasing by 0.27% of the total amount available can also be attributed to the demanding SSR arrays. Area usage can be dramatically reduced by un-partitioning the SSR

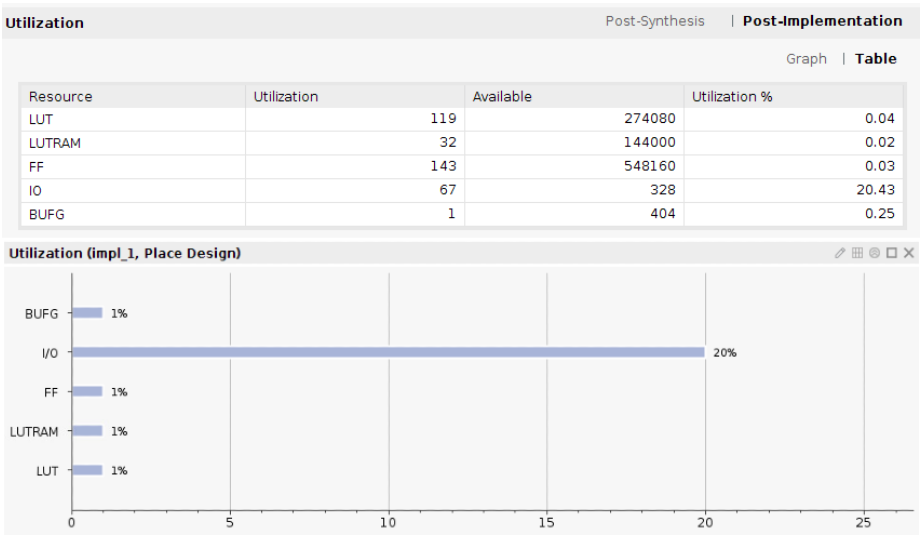

Figure 9: This figure details the resource statistics for the previously built pedestal subtraction algorithm, with the SSR mechanism included in the statistics. This is the result of the block being written from scratch in VHDL. This figure enables a comparison to the statistics of the `pedsub_HLS_SSR` block

memories and pursuing the methods described in section 3.2.1 so as to avoid the demanding requirement for latency = II = 1. This would use a small number of BRAMs instead of large numbers of LUTs and FFs, the analogue of which is seen in the original design requiring 32 LUTRAMs.

3.2 Finite Impulse Response Filter

The FIR algorithm acts as a random noise filter for the pedestal-subtracted ADC values, so that the final signal entering the hit finder block is clean enough not to generate false hits.

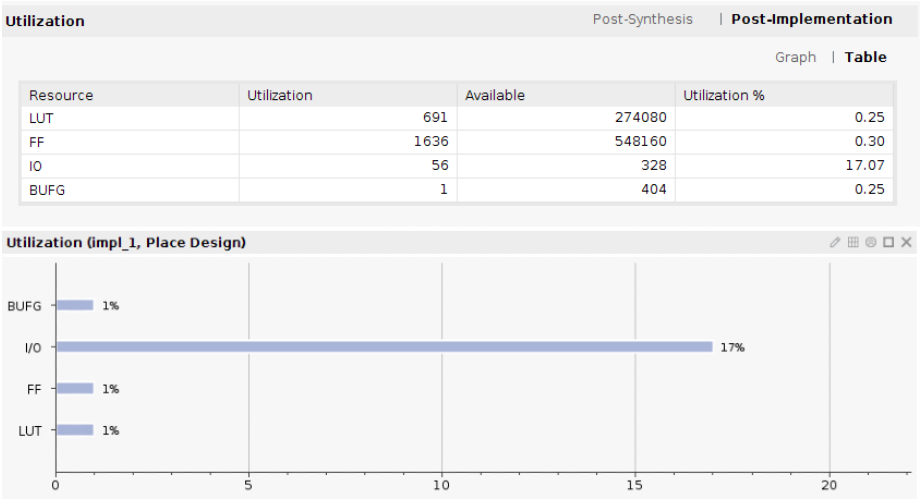

Figure 10: This figure details the resource statistics for the pedestal subtraction algorithm generated from HLS with an SSR memory built into the design, to be compared to the original `PedSub_SSR` block written directly in VHDL.

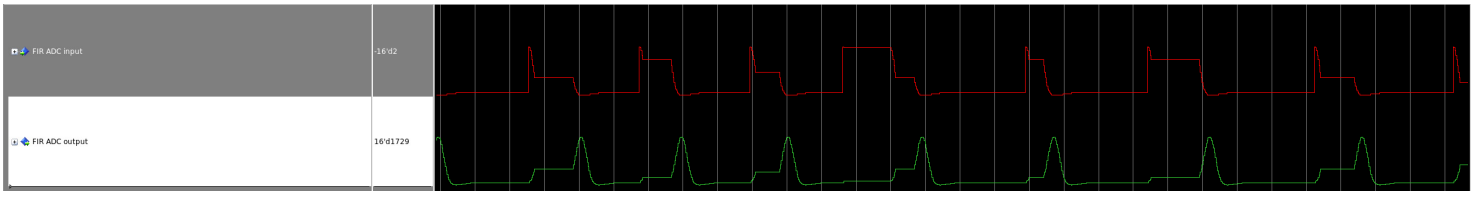

Figure 11: Detailed in the graphic are the analogue ADC input and output signals generated in a Questasim simulation. Note the clear peaks where the hits occur in the green output below, in contrast to the unfiltered red input above. Plateaus in the signal are a result of the random back pressure events pausing ADC input and output.

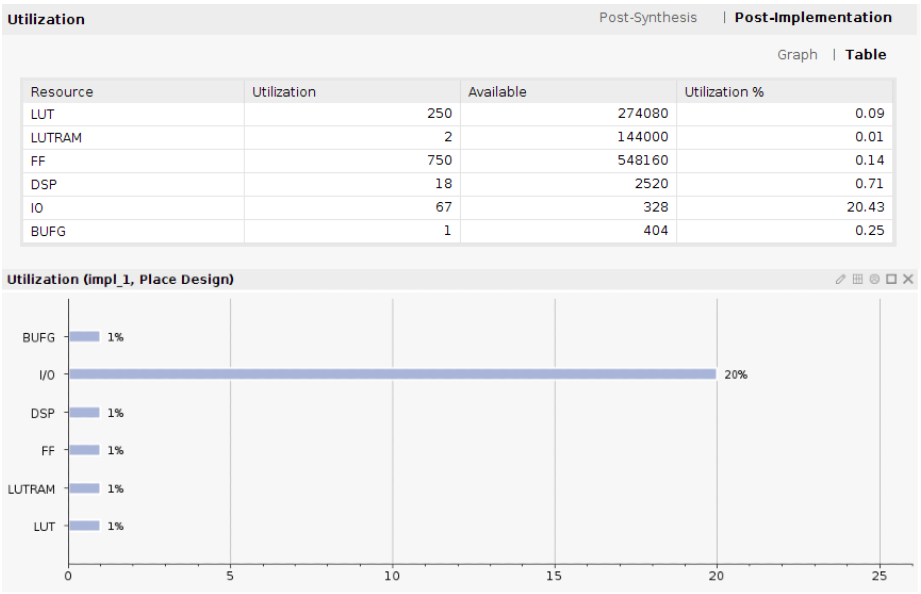

Figure 12: This figure details the resource statistics of the previously written FIR filter algorithm with the SSR mechanism not included, to allow for comparison to the analagous redesigned HLS module.

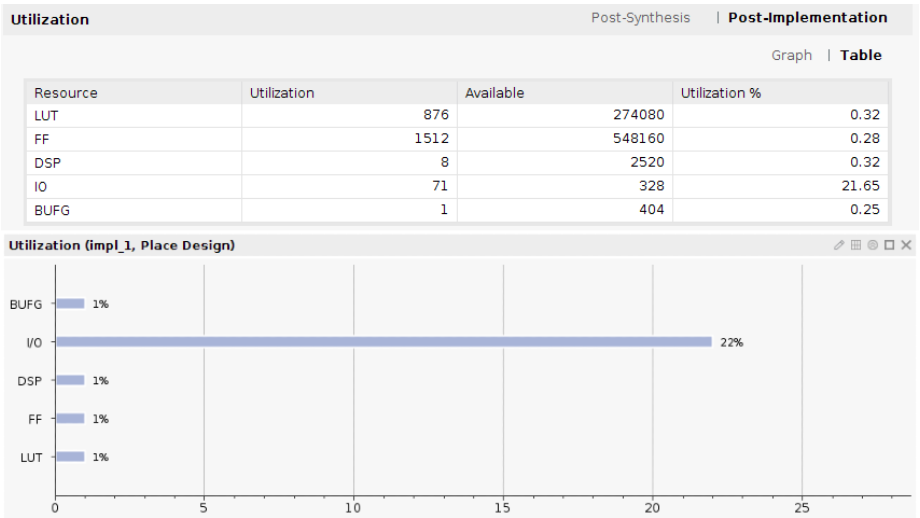

Figure 13: This figure contains the resource statistics for the FIR filter algorithm written in HLS, connected to a previously written external SSR mechanism that is not included in the statistics. The only resources being spent on the SSR are those allocated for accepting an storing the output values from the external SSR block.

Resource	Utilization	Available	Utilization %
LUT	261	274080	0.10
LUTRAM	2	144000	0.01
FF	747	548160	0.14
BRAM	1	912	0.11
DSP	18	2520	0.71
IO	42	328	12.80
BUFG	1	404	0.25

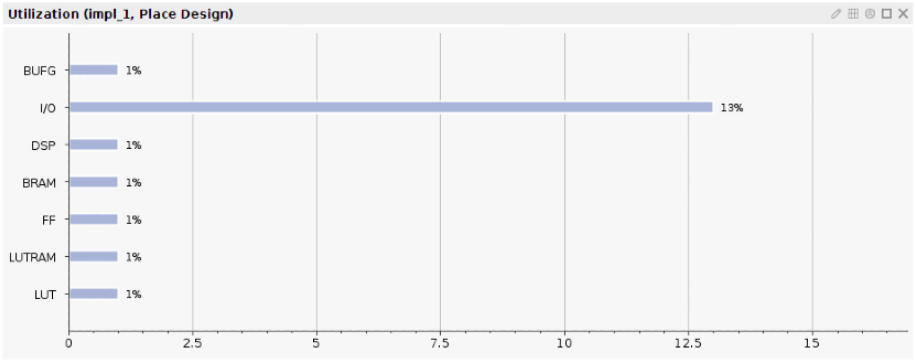

Figure 14: This figure shows the resource statistics for the previously built FIR algorithm with the SSR mechanism included in the usage. This is included to allow the comparison to the HLS FIR block with an in-built SSR mechanism.

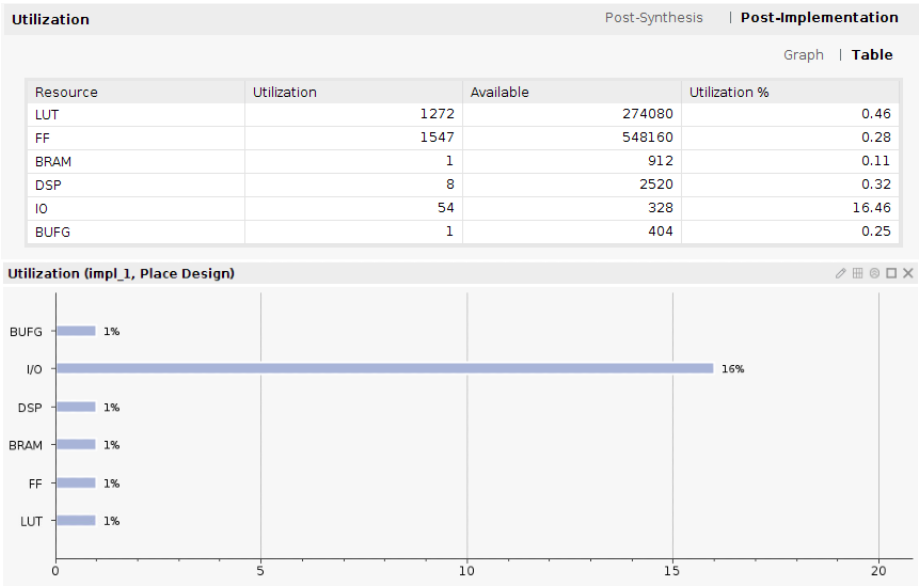

Figure 15: The above graphic demonstrates the resource statistics for the HLS-synthesized FIR algorithm with an in-built SSR memory and access mechanism, to enable the comparison to the version written from scratch in a HDL.

3.2.1 Algorithm Operations

AXI4 Stream Protocol Directive Method for Back Pressure Behaviour

3.2.2 Redesign of FIR with External SSR

Timing and Accommodating the Tap Restoration

Performance and Resource Utilization Comparison

3.2.3 FIR with Implemented SSR

3.2.4 Optimising for Back Pressure Functionality

Performance and Resource Utilization Comparison

4 Design Processs Conclusions

4.1 Simulation using Questasim

4.2 Timeline analysis

4.3 Final Thoughts

5 Future of High Level Synthesis and the DUNE Experiment

References

[1] "Dune science home page." `https://www.dunescience.org/`, 2020. Accessed 11-01-2021.

[2] S. Mertens, "Direct neutrino mass experiments," *Journal of Physics: Conference Series*, vol. 718, p. 022013, May 2016.

[3] P. S. Cooper, "The masses of the neutrinos," *arXiv preprint arXiv:1605.03159*, 2016.

[4] R. Acciarri, M. A. Acero, M. Adamowski, C. Adams, P. Adamson, S. Adhikari, *et al.*, "Long-baseline neutrino facility (lbnf) and deep underground neutrino experiment (dune) conceptual design report volume 1: The lbnf and dune projects," 2016.

[5] M. Kobayashi and T. Maskawa, "Cp-violation in the renormalizable theory of weak interaction," *Progress of theoretical physics*, vol. 49, no. 2, pp. 652–657, 1973.

[6] P. Di Bari, "An introduction to leptogenesis and neutrino properties," *Contemporary Physics*, vol. 53, p. 315–338, Jul 2012.

[7] B. Abi, G. Barr, J. Martin-Albo, and F. Azfar, "Dune's daq architecture; envisaging the alternatives," Aug 2017.

[8] C. Rubbia, "The liquid-argon time projection chamber: a new concept for neutrino detectors," tech. rep., 1977.

[9] C. Adams, M. Alrashed, R. An, J. Anthony, J. Asaadi, A. Ashkenazi, M. Auger, S. Balasubramanian, B. Baller, C. Barnes, *et al.*, "Rejecting cosmic background for exclusive neutrino interaction studies with liquid argon tpcs; a case study with the microboone detector," *arXiv preprint arXiv:1812.05679*, 2018.

[10] T. Collaboration, B. Abi, R. Acciarri, M. Acero, M. Adamowski, C. Adams, D. Adams, P. Adamson, M. Adinolfi, Z. Ahmad, C. Albright, T. Alion, J. Anderson, K. Anderson, C. Andreopoulos, M. Andrews, R. Andrews, J. Anjos, A. Ankowski, and R. Zwaska, "The single-phase protodune technical design report," 06 2017.

[11] B. Aimard, C. Alt, J. Asaadi, M. Auger, V. Aushev, D. Autiero, M. Badoi, A. Balaceanu, G. Balik, L. Balleyguier, *et al.*, "A 4 tonne demonstrator for large-scale dual-phase liquid argon time projection chambers," *Journal of Instrumentation*, vol. 13, no. 11, p. P11003, 2018.

[12] A. Borga, E. Church, F. Filthaut, E. Gamberini, R. Habraken, G. L. Miotto, T. Miedema, F. Schreuder, J. Schumacher, R. Sipos, *et al.*, "Felix based readout of the single-phase protodune detector," in *EPJ Web of Conferences*, vol. 214, p. 01013, EDP Sciences, 2019.

[13] D. Adams, M. Bass, M. Bishai, C. Bromberg, J. Calcutt, H. Chen, J. Fried, I. Furic, S. Gao, D. Gastler, *et al.*, "The protodune-sp lartpc electronics production, commissioning, and performance," *Journal of Instrumentation*, vol. 15, no. 06, p. P06017, 2020.

[14] K. Manolopoulos, "Trigger primitive generation - firmware." https://indico.fnal.gov/event/44240/contributions/190328/attachments/132052/162111/UPDAQ_TechRev_TPGfw.pdf, Jul 2020.

[15] P. Coussy, D. D. Gajski, M. Meredith, and A. Takach, "An introduction to high-level synthesis," *IEEE Design & Test of Computers*, vol. 26, no. 4, pp. 8–17, 2009.

[16] "Vivado design suite user guide 2018." https://www.xilinx.com/support/documentation/sw_manuals/xilinx2017_4/ug902-vivado-high-level-synthesis.pdf, Dec 2018. Xilinx.

[17] F. Vahid, *Digital design with RTL design, VHDL, and Verilog*. John Wiley & Sons, 2010.

[18] S. K. Bose, *Hardware and Software of Personal Computers*. New Age International, 1996.

[19] M. Hall and J. P. McNamee, "Improving software performance with automatic memoization," *Johns Hopkins APL Technical Digest*, vol. 18, no. 2, p. 255, 1997.

[20] B. G. Liptak, *Process Control and Optimization. Instrument Engineers' Handbook, Volume II.* CRC Press, 2006.

[21] C. Lo and P. Chow, "Model-based optimization of high level synthesis directives," in *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*, pp. 1–10, IEEE, 2016.